Best Practices Guide - E-commerce Cart System

Table of Contents

1. Development Best Practices
2. Code Quality Standards
3. Testing Best Practices
4. Documentation Standards
5. Version Control Guidelines
6. Security Best Practices
7. Performance Guidelines
8. Deployment Best Practices

Development Best Practices

1. Code Organization and Structure

Project Structure

```
src/
├── domain/                    # Domain layer (business logic)
│   ├── entities/              # Domain entities
│   ├── value-objects/         # Value objects
│   ├── services/              # Domain services
│   ├── events/                # Domain events
│   └── repositories/          # Repository interfaces
├── application/               # Application layer (use cases)
│   ├── commands/              # Command handlers
│   ├── queries/               # Query handlers
│   ├── services/              # Application services
│   └── dtos/                  # Data transfer objects
├── infrastructure/            # Infrastructure layer
│   ├── persistence/           # Repository implementations
│   ├── external/              # External service adapters
│   ├── messaging/             # Message brokers
│   ├── security/              # Authentication/authorization
│   └── logging/               # Logging implementations
└── interfaces/                # Interface layer
    ├── rest/                  # REST API controllers
    ├── graphql/               # GraphQL resolvers
    └── web/                   # Web interface
```

File Naming Conventions

```
// Entities: PascalCase
Product.ts
Order.ts
User.ts

// Value Objects: PascalCase
ProductId.ts
Money.ts
Email.ts

// Services: PascalCase with "Service" suffix
OrderService.ts
PaymentService.ts
NotificationService.ts

// Repositories: PascalCase with "Repository" suffix
ProductRepository.ts
OrderRepository.ts

// Interfaces: PascalCase with "I" prefix
IProductRepository.ts
IOrderService.ts

// Controllers: PascalCase with "Controller" suffix
ProductController.ts
OrderController.ts

// DTOs: PascalCase with "Dto" suffix
CreateProductDto.ts
UpdateOrderDto.ts
```

2. Coding Standards

TypeScript Best Practices

```
// ☑ Use strict TypeScript configuration
{
  "compilerOptions": {
    "strict": true,
    "noImplicitAny": true,
    "noImplicitReturns": true,
    "noUnusedLocals": true,
    "noUnusedParameters": true
  }
}

// ☑ Use interfaces for object shapes
interface Product {
  readonly id: ProductId;
  readonly name: ProductName;
```

```
  readonly price: Money;
  readonly description: ProductDescription;
}

// ☑ Use type aliases for complex types
type OrderStatus = 'pending' | 'confirmed' | 'shipped' | 'delivered' |
'cancelled';
type PaymentMethod = 'credit_card' | 'paypal' | 'bank_transfer';

// ☑ Use enums for constants
enum Currency {
  USD = 'USD',
  EUR = 'EUR',
  GBP = 'GBP'
}

// ☑ Use readonly for immutable properties
class Product {
  constructor(
    private readonly id: ProductId,
    private readonly name: ProductName,
    private price: Money // Mutable property
  ) {}
}

// ☑ Use proper error handling
class ProductService {
  async findById(id: ProductId): Promise<Result<Product, ProductNotFoundError>> {
    try {
      const product = await this.repository.findById(id);
      if (!product) {
        return Result.failure(new ProductNotFoundError(id));
      }
      return Result.success(product);
    } catch (error) {
      return Result.failure(new DatabaseError(error.message));
    }
  }
}
```

Error Handling

```
// ☑ Use custom error classes
class DomainError extends Error {
  constructor(message: string) {
    super(message);
    this.name = 'DomainError';
  }
}

class ProductNotFoundError extends DomainError {
```

```
    constructor(productId: ProductId) {
      super(`Product with id ${productId.value} not found`);
      this.name = 'ProductNotFoundError';
    }
  }

  // ☑ Use Result pattern for error handling
  class Result<T, E> {
    private constructor(
      private readonly isSuccess: boolean,
      private readonly value?: T,
      private readonly error?: E
    ) {}

    static success<T, E>(value: T): Result<T, E> {
      return new Result<T, E>(true, value, undefined);
    }

    static failure<T, E>(error: E): Result<T, E> {
      return new Result<T, E>(false, undefined, error);
    }

    getValue(): T {
      if (!this.isSuccess) {
        throw new Error('Cannot get value from failure result');
      }
      return this.value!;
    }

    getError(): E {
      if (this.isSuccess) {
        throw new Error('Cannot get error from success result');
      }
      return this.error!;
    }

    isFailure(): boolean {
      return !this.isSuccess;
    }
  }
```

3. Domain-Driven Design Practices

Entity Design

```
  // ☑ Rich domain models with business logic
  class Order {
    private constructor(
      private readonly id: OrderId,
      private readonly orderNumber: OrderNumber,
      private customer: Customer,
```

```
    private items: OrderItem[],
    private totalAmount: Money,
    private status: OrderStatus,
    private readonly createdAt: Date,
    private updatedAt: Date
  ) {}

  // Factory method
  static create(customer: Customer, items: OrderItem[]): Order {
    const orderNumber = OrderNumber.generate();
    const totalAmount = this.calculateTotal(items);

    return new Order(
      OrderId.generate(),
      orderNumber,
      customer,
      items,
      totalAmount,
      OrderStatus.PENDING,
      new Date(),
      new Date()
    );
  }

  // Business methods
  addItem(product: Product, quantity: Quantity): void {
    if (this.status !== OrderStatus.PENDING) {
      throw new DomainError('Cannot modify confirmed order');
    }

    const existingItem = this.items.find(item =>
      item.productId.equals(product.id)
    );

    if (existingItem) {
      existingItem.updateQuantity(existingItem.quantity.add(quantity));
    } else {
      const newItem = OrderItem.create(product, quantity);
      this.items.push(newItem);
    }

    this.recalculateTotal();
    this.updatedAt = new Date();
  }

  confirm(): void {
    if (this.status !== OrderStatus.PENDING) {
      throw new DomainError('Order can only be confirmed from pending status');
    }
    this.status = OrderStatus.CONFIRMED;
    this.updatedAt = new Date();
  }

  private recalculateTotal(): void {
```

```
    this.totalAmount = this.items.reduce(
      (total, item) => total.add(item.subtotal),
      Money.zero()
    );
  }

  private static calculateTotal(items: OrderItem[]): Money {
    return items.reduce(
      (total, item) => total.add(item.subtotal),
      Money.zero()
    );
  }
}
```

Value Object Design

```
// ☑ Immutable value objects
class Money {
  private constructor(
    private readonly amount: number,
    private readonly currency: Currency
  ) {
    if (amount < 0) {
      throw new DomainError('Money amount cannot be negative');
    }
  }

  add(other: Money): Money {
    if (this.currency !== other.currency) {
      throw new DomainError('Cannot add money with different currencies');
    }
    return new Money(this.amount + other.amount, this.currency);
  }

  multiply(factor: number): Money {
    return new Money(this.amount * factor, this.currency);
  }

  static zero(currency: Currency = Currency.USD): Money {
    return new Money(0, currency);
  }

  static fromNumber(amount: number, currency: Currency = Currency.USD): Money {
    return new Money(amount, currency);
  }

  get value(): number {
    return this.amount;
  }

  equals(other: Money): boolean {
```

```
    return this.amount === other.amount && this.currency === other.currency;
  }

  toString(): string {
    return `${this.currency} ${this.amount.toFixed(2)}`;
  }
}
```

🔧 Code Quality Standards

1. Code Review Checklist

General Code Quality

- [ ] Code follows established naming conventions
- [ ] Functions and methods are small and focused
- [ ] No code duplication (DRY principle)
- [ ] Proper error handling and validation
- [ ] Meaningful variable and function names
- [ ] Comments explain "why" not "what"
- [ ] No hardcoded values (use constants)

TypeScript Specific

- [ ] Proper use of types and interfaces
- [ ] No `any` types without justification
- [ ] Use of readonly where appropriate
- [ ] Proper use of generics
- [ ] No unused imports or variables
- [ ] Proper use of access modifiers

Domain Logic

- [ ] Business rules are encapsulated in domain entities
- [ ] No business logic in controllers or repositories
- [ ] Proper use of value objects
- [ ] Domain events for side effects
- [ ] Proper validation at domain boundaries

2. Linting and Formatting

ESLint Configuration

```
{
  "extends": [
    "@typescript-eslint/recommended",
    "@typescript-eslint/recommended-requiring-type-checking"
```

```
    ],
    "rules": {
      "@typescript-eslint/no-unused-vars": "error",
      "@typescript-eslint/explicit-function-return-type": "warn",
      "@typescript-eslint/no-explicit-any": "error",
      "@typescript-eslint/prefer-readonly": "warn",
      "prefer-const": "error",
      "no-var": "error"
    }
  }
```

Prettier Configuration

```
{
  "semi": true,
  "trailingComma": "es5",
  "singleQuote": true,
  "printWidth": 80,
  "tabWidth": 2,
  "useTabs": false
}
```

3. Code Metrics

Complexity Guidelines

- **Cyclomatic Complexity**: Maximum 10 per function
- **Lines of Code**: Maximum 50 per function
- **Parameters**: Maximum 5 per function
- **Nesting Levels**: Maximum 3 levels deep

Code Coverage Targets

- **Unit Tests**: Minimum 80% coverage
- **Integration Tests**: Minimum 70% coverage
- **Critical Paths**: 100% coverage

🧪 Testing Best Practices

1. Test Structure

Test Organization

```
tests/
├── unit/                    # Unit tests
│   ├── domain/             # Domain entity tests
```

```
│   ├── application/         # Use case tests
│   └── infrastructure/      # Repository tests
├── integration/             # Integration tests
│   ├── api/                 # API endpoint tests
│   ├── database/            # Database integration tests
│   └── external/            # External service tests
└── e2e/                     # End-to-end tests
    ├── user-journeys/       # User journey tests
    └── critical-flows/      # Critical business flows
```

Test Naming Convention

```
// Unit test naming: describe('ClassName', () => { describe('methodName', () => {
it('should do something when condition', () => {}) }) })
describe('Order', () => {
  describe('addItem', () => {
    it('should add new item to order when item does not exist', () => {
      // Test implementation
    });

    it('should update quantity when item already exists', () => {
      // Test implementation
    });

    it('should throw error when order is not in pending status', () => {
      // Test implementation
    });
  });
});
```

2. Unit Testing

Domain Entity Testing

```
describe('Product', () => {
  let product: Product;
  let validName: ProductName;
  let validPrice: Money;

  beforeEach(() => {
    validName = new ProductName('Test Product');
    validPrice = Money.fromNumber(99.99);
    product = Product.create(validName, validPrice, Category.ELECTRONICS);
  });

  describe('updatePrice', () => {
    it('should update price when new price is valid', () => {
      const newPrice = Money.fromNumber(149.99);
```

```
      product.updatePrice(newPrice);
      expect(product.price).toEqual(newPrice);
    });

    it('should throw error when new price is negative', () => {
      const negativePrice = Money.fromNumber(-10);
      expect(() => product.updatePrice(negativePrice))
        .toThrow(DomainError);
    });
  });

  describe('reserveStock', () => {
    beforeEach(() => {
      product.updateStock(new StockQuantity(100));
    });

    it('should reduce stock when quantity is available', () => {
      const quantity = new StockQuantity(10);
      product.reserveStock(quantity);
      expect(product.stockQuantity.value).toBe(90);
    });

    it('should throw error when insufficient stock', () => {
      const quantity = new StockQuantity(150);
      expect(() => product.reserveStock(quantity))
        .toThrow(DomainError);
    });
  });
});
```

Use Case Testing

```
describe('CreateOrderUseCase', () => {
  let useCase: CreateOrderUseCase;
  let mockOrderRepository: jest.Mocked<IOrderRepository>;
  let mockProductRepository: jest.Mocked<IProductRepository>;
  let mockPaymentService: jest.Mocked<IPaymentService>;

  beforeEach(() => {
    mockOrderRepository = createMockOrderRepository();
    mockProductRepository = createMockProductRepository();
    mockPaymentService = createMockPaymentService();

    useCase = new CreateOrderUseCase(
      mockOrderRepository,
      mockProductRepository,
      mockPaymentService
    );
  });

  it('should create order successfully when all conditions are met', async () => {
```

```
    // Arrange
    const command = new CreateOrderCommand(
      customerId,
      [{ productId, quantity }]
    );

    const product = createMockProduct();
    mockProductRepository.findById.mockResolvedValue(product);
    mockOrderRepository.save.mockResolvedValue();

    // Act
    const result = await useCase.execute(command);

    // Assert
    expect(result.isSuccess()).toBe(true);
    expect(mockOrderRepository.save).toHaveBeenCalledWith(
      expect.objectContaining({
        customerId: command.customerId,
        items: expect.arrayContaining([
          expect.objectContaining({
            productId: command.items[0].productId,
            quantity: command.items[0].quantity
          })
        ])
      })
    );
  });

  it('should fail when product is not found', async () => {
    // Arrange
    const command = new CreateOrderCommand(customerId, [{ productId, quantity }]);
    mockProductRepository.findById.mockResolvedValue(null);

    // Act
    const result = await useCase.execute(command);

    // Assert
    expect(result.isFailure()).toBe(true);
    expect(result.getError()).toBeInstanceOf(ProductNotFoundError);
  });
});
```

3. Integration Testing

API Endpoint Testing

```
describe('ProductController', () => {
  let app: Express;
  let mockProductService: jest.Mocked<IProductService>;

  beforeEach(async () => {
```

```
    mockProductService = createMockProductService();

    app = express();
    app.use('/api/products', productRouter(mockProductService));
  });

  describe('GET /api/products/:id', () => {
    it('should return product when found', async () => {
      // Arrange
      const productId = '123';
      const product = createMockProduct();
      mockProductService.findById.mockResolvedValue(Result.success(product));

      // Act
      const response = await request(app)
        .get(`/api/products/${productId}`)
        .expect(200);

      // Assert
      expect(response.body).toEqual({
        id: product.id.value,
        name: product.name.value,
        price: product.price.value,
        description: product.description.value
      });
    });

    it('should return 404 when product not found', async () => {
      // Arrange
      const productId = '999';
      mockProductService.findById.mockResolvedValue(
        Result.failure(new ProductNotFoundError(new ProductId(productId)))
      );

      // Act & Assert
      await request(app)
        .get(`/api/products/${productId}`)
        .expect(404);
    });
  });
});
```

4. Test Data Management

Test Factories

```
class ProductFactory {
  static create(overrides: Partial<ProductProps> = {}): Product {
    const defaults: ProductProps = {
      id: ProductId.generate(),
      name: new ProductName('Test Product'),
```

```
      description: new ProductDescription('Test description'),
      price: Money.fromNumber(99.99),
      stockQuantity: new StockQuantity(100),
      category: Category.ELECTRONICS,
      isActive: true,
      createdAt: new Date(),
      updatedAt: new Date()
    };

    return new Product({ ...defaults, ...overrides });
  }

  static createInactive(): Product {
    return this.create({ isActive: false });
  }

  static createOutOfStock(): Product {
    return this.create({ stockQuantity: new StockQuantity(0) });
  }
}
```

Documentation Standards

1. Code Documentation

JSDoc Comments

```
/**
 * Represents a product in the e-commerce system.
 *
 * Products are the core entities that customers can browse and purchase.
 * Each product belongs to a category and has inventory tracking.
 *
 * @example
 * ```typescript
 * const product = Product.create(
 *   new ProductName('iPhone 13'),
 *   Money.fromNumber(999.99),
 *   Category.ELECTRONICS
 * );
 * ```
 */
class Product {
  /**
   * Updates the product price.
   *
   * @param newPrice - The new price for the product
   * @throws {DomainError} When the new price is negative
   *
   * @example
```

```
   * ```typescript
   * product.updatePrice(Money.fromNumber(899.99));
   * ```
   */
  updatePrice(newPrice: Money): void {
    if (newPrice.isNegative()) {
      throw new DomainError('Price cannot be negative');
    }
    this.price = newPrice;
    this.updatedAt = new Date();
  }

  /**
   * Checks if the product is available for purchase.
   *
   * A product is available when it's active and has stock.
   *
   * @returns True if the product is available, false otherwise
   *
   * @example
   * ```typescript
   * if (product.isAvailable()) {
   *   // Add to cart
   * }
   * ```
   */
  isAvailable(): boolean {
    return this.isActive &&
this.stockQuantity.isGreaterThan(StockQuantity.zero());
  }
}
```

API Documentation

```
/**
 * @api {get} /api/products/:id Get Product
 * @apiName GetProduct
 * @apiGroup Products
 * @apiVersion 1.0.0
 *
 * @apiParam {String} id Product unique ID
 *
 * @apiSuccess {String} id Product ID
 * @apiSuccess {String} name Product name
 * @apiSuccess {Number} price Product price
 * @apiSuccess {String} description Product description
 * @apiSuccess {String} category Product category
 * @apiSuccess {Number} stockQuantity Available stock
 * @apiSuccess {Boolean} isActive Product availability
 *
 * @apiSuccessExample {json} Success-Response:
```

```
 *     HTTP/1.1 200 OK
 *     {
 *       "id": "123",
 *       "name": "iPhone 13",
 *       "price": 999.99,
 *       "description": "Latest iPhone model",
 *       "category": "electronics",
 *       "stockQuantity": 50,
 *       "isActive": true
 *     }
 *
 * @apiError ProductNotFound The requested product was not found
 *
 * @apiErrorExample {json} Error-Response:
 *     HTTP/1.1 404 Not Found
 *     {
 *       "error": "Product not found",
 *       "message": "Product with id 999 does not exist"
 *     }
 */
```

2. README Documentation

Project README Template

```
# E-commerce Cart System

## Overview
Brief description of the system and its purpose.

## Features
- Product catalog management
- Shopping cart functionality
- Order processing
- User management
- Payment processing

## Technology Stack
- **Backend**: Node.js, TypeScript, Express
- **Database**: PostgreSQL
- **Frontend**: React, TypeScript
- **Testing**: Jest, Supertest
- **Documentation**: JSDoc, Swagger

## Getting Started

### Prerequisites
- Node.js 18+
- PostgreSQL 14+
- Docker (optional)
```

```
### Installation
```bash
# Clone the repository
git clone https://github.com/your-org/ecommerce-cart.git

# Install dependencies
npm install

# Set up environment variables
cp .env.example .env

# Run database migrations
npm run migrate

# Start development server
npm run dev
```

Environment Variables

```
DATABASE_URL=postgresql://user:password@localhost:5432/ecommerce
JWT_SECRET=your-jwt-secret
PAYMENT_API_KEY=your-payment-api-key
```

API Documentation

API documentation is available at `/api/docs` when running the server.

Testing

```
# Run unit tests
npm run test:unit

# Run integration tests
npm run test:integration

# Run all tests with coverage
npm run test:coverage
```

Contributing

Please read CONTRIBUTING.md for details on our code of conduct and the process for submitting pull requests.

License

This project is licensed under the MIT License - see the LICENSE file for details.
```

```
---

## 🔄 Version Control Guidelines

### 1. Git Workflow

#### Branch Naming Convention
```

feature/user-stories-123-add-product-search bugfix/order-processing-456-fix-payment-validation hotfix/security-789-fix-sql-injection release/v1.2.0

```
#### Commit Message Format
```

():

[optional body]

[optional footer]

```
**Types:**
- `feat`: New feature
- `fix`: Bug fix
- `docs`: Documentation changes
- `style`: Code style changes (formatting, etc.)
- `refactor`: Code refactoring
- `test`: Adding or updating tests
- `chore`: Maintenance tasks

**Examples:**
```

feat(product): add product search functionality

- Implement product search by name and description
- Add search filters by category and price range
- Include search result pagination

Closes #123

```

```

fix(order): resolve payment validation issue

- Fix credit card validation logic
- Add proper error handling for invalid cards
- Update payment error messages

Fixes #456

```
### 2. Pull Request Guidelines

#### PR Template
```markdown
## Description
Brief description of the changes made.

## Type of Change
- [ ] Bug fix (non-breaking change which fixes an issue)
- [ ] New feature (non-breaking change which adds functionality)
- [ ] Breaking change (fix or feature that would cause existing functionality to
not work as expected)
- [ ] Documentation update

## Testing
- [ ] Unit tests pass
- [ ] Integration tests pass
- [ ] Manual testing completed
- [ ] Code coverage maintained

## Checklist
- [ ] Code follows style guidelines
- [ ] Self-review completed
- [ ] Documentation updated
- [ ] No console.log statements
- [ ] No hardcoded values

## Related Issues
Closes #123
```

Code Review Checklist

- ☐ Code follows established patterns and conventions
- ☐ All tests pass and coverage is maintained
- ☐ No security vulnerabilities introduced
- ☐ Performance impact considered
- ☐ Documentation updated if needed
- ☐ Error handling is appropriate
- ☐ No debugging code left in

Security Best Practices
```

1. Input Validation

Validation Middleware

```
import { body, validationResult } from 'express-validator';

const validateProduct = [
  body('name')
    .trim()
    .isLength({ min: 3, max: 200 })
    .withMessage('Product name must be between 3 and 200 characters'),

  body('price')
    .isFloat({ min: 0.01 })
    .withMessage('Price must be a positive number'),

  body('categoryId')
    .isUUID()
    .withMessage('Category ID must be a valid UUID'),

  (req: Request, res: Response, next: NextFunction) => {
    const errors = validationResult(req);
    if (!errors.isEmpty()) {
      return res.status(400).json({
        errors: errors.array()
      });
    }
    next();
  }
];
```

SQL Injection Prevention

```
// ☑ Use parameterized queries
class ProductRepository {
  async findById(id: ProductId): Promise<Product | null> {
    const result = await this.connection.query(
      'SELECT * FROM products WHERE id = $1',
      [id.value]
    );
    return result.rows[0] ? this.mapToProduct(result.rows[0]) : null;
  }

  async searchByName(name: string): Promise<Product[]> {
    const result = await this.connection.query(
      'SELECT * FROM products WHERE name ILIKE $1',
      [`%${name}%`]
    );
    return result.rows.map(row => this.mapToProduct(row));
```

```
  }
}
```

2. Authentication and Authorization

JWT Implementation

```
import jwt from 'jsonwebtoken';

class JwtService {
  private readonly secret: string;
  private readonly expiresIn: string;

  constructor() {
    this.secret = process.env.JWT_SECRET!;
    this.expiresIn = '24h';
  }

  generateToken(payload: TokenPayload): string {
    return jwt.sign(payload, this.secret, {
      expiresIn: this.expiresIn,
      issuer: 'ecommerce-api',
      audience: 'ecommerce-users'
    });
  }

  verifyToken(token: string): TokenPayload {
    try {
      return jwt.verify(token, this.secret, {
        issuer: 'ecommerce-api',
        audience: 'ecommerce-users'
      }) as TokenPayload;
    } catch (error) {
      throw new AuthenticationError('Invalid token');
    }
  }
}
```

Role-Based Access Control

```
const requireRole = (requiredRole: string) => {
  return (req: Request, res: Response, next: NextFunction) => {
    const user = req.user as User;

    if (!user) {
      return res.status(401).json({ error: 'Authentication required' });
    }
```

```
    const hasRole = user.roles.some(role => role.name === requiredRole);
    if (!hasRole) {
      return res.status(403).json({ error: 'Insufficient permissions' });
    }

    next();
  };
};

// Usage
app.get('/api/admin/products',
  authenticate,
  requireRole('admin'),
  productController.getAll
);
```

3. Data Protection

Password Hashing

```
import bcrypt from 'bcrypt';

class PasswordService {
  private readonly saltRounds = 12;

  async hashPassword(password: string): Promise<string> {
    return bcrypt.hash(password, this.saltRounds);
  }

  async verifyPassword(password: string, hash: string): Promise<boolean> {
    return bcrypt.compare(password, hash);
  }
}
```

Data Encryption

```
import crypto from 'crypto';

class EncryptionService {
  private readonly algorithm = 'aes-256-gcm';
  private readonly key: Buffer;

  constructor() {
    this.key = Buffer.from(process.env.ENCRYPTION_KEY!, 'hex');
  }

  encrypt(text: string): { encryptedData: string; iv: string; authTag: string } {
    const iv = crypto.randomBytes(16);
```

```
    const cipher = crypto.createCipher(this.algorithm, this.key, iv);

    let encrypted = cipher.update(text, 'utf8', 'hex');
    encrypted += cipher.final('hex');

    return {
      encryptedData: encrypted,
      iv: iv.toString('hex'),
      authTag: cipher.getAuthTag().toString('hex')
    };
  }

  decrypt(encryptedData: string, iv: string, authTag: string): string {
    const decipher = crypto.createDecipher(
      this.algorithm,
      this.key,
      Buffer.from(iv, 'hex')
    );

    decipher.setAuthTag(Buffer.from(authTag, 'hex'));

    let decrypted = decipher.update(encryptedData, 'hex', 'utf8');
    decrypted += decipher.final('utf8');

    return decrypted;
  }
}
```

⚡ Performance Guidelines

1. Database Optimization

Query Optimization

```
// ☑ Use proper indexing
CREATE INDEX idx_product_category_price ON products(category_id, price);
CREATE INDEX idx_order_customer_status ON orders(customer_id, status);
CREATE INDEX idx_order_created_at ON orders(created_at DESC);

// ☑ Use pagination for large datasets
class ProductRepository {
  async findWithPagination(
    page: number = 1,
    limit: number = 20,
    filters?: ProductFilters
  ): Promise<PaginatedResult<Product>> {
    const offset = (page - 1) * limit;

    const query = `
      SELECT p.*, c.name as category_name
```

```
      FROM products p
      JOIN categories c ON p.category_id = c.id
      WHERE p.is_active = true
      ${filters?.categoryId ? 'AND p.category_id = $3' : ''}
      ${filters?.minPrice ? 'AND p.price >= $4' : ''}
      ${filters?.maxPrice ? 'AND p.price <= $5' : ''}
      ORDER BY p.created_at DESC
      LIMIT $1 OFFSET $2
    `;

    const params = [limit, offset, ...this.buildFilterParams(filters)];
    const result = await this.connection.query(query, params);

    return {
      data: result.rows.map(row => this.mapToProduct(row)),
      pagination: {
        page,
        limit,
        total: await this.getTotalCount(filters),
        totalPages: Math.ceil(total / limit)
      }
    };
  }
}
```

Connection Pooling

```
import { Pool } from 'pg';

class DatabaseConnection {
  private pool: Pool;

  constructor() {
    this.pool = new Pool({
      host: process.env.DB_HOST,
      port: parseInt(process.env.DB_PORT!),
      database: process.env.DB_NAME,
      user: process.env.DB_USER,
      password: process.env.DB_PASSWORD,
      max: 20, // Maximum number of clients
      idleTimeoutMillis: 30000,
      connectionTimeoutMillis: 2000,
    });
  }

  async query(text: string, params?: any[]): Promise<QueryResult> {
    const client = await this.pool.connect();
    try {
      return await client.query(text, params);
    } finally {
      client.release();
```

```
    }
  }
}
```

2. Caching Strategy

Redis Caching

```
import Redis from 'ioredis';

class CacheService {
  private redis: Redis;

  constructor() {
    this.redis = new Redis({
      host: process.env.REDIS_HOST,
      port: parseInt(process.env.REDIS_PORT!),
      password: process.env.REDIS_PASSWORD,
    });
  }

  async get<T>(key: string): Promise<T | null> {
    const value = await this.redis.get(key);
    return value ? JSON.parse(value) : null;
  }

  async set(key: string, value: any, ttl: number = 3600): Promise<void> {
    await this.redis.setex(key, ttl, JSON.stringify(value));
  }

  async invalidate(pattern: string): Promise<void> {
    const keys = await this.redis.keys(pattern);
    if (keys.length > 0) {
      await this.redis.del(...keys);
    }
  }
}

// Usage in repository
class ProductRepository {
  constructor(
    private db: DatabaseConnection,
    private cache: CacheService
  ) {}

  async findById(id: ProductId): Promise<Product | null> {
    const cacheKey = `product:${id.value}`;

    // Try cache first
    const cached = await this.cache.get<Product>(cacheKey);
    if (cached) {
```

```
      return this.mapToProduct(cached);
    }

    // Query database
    const result = await this.db.query(
      'SELECT * FROM products WHERE id = $1',
      [id.value]
    );

    if (result.rows[0]) {
      const product = this.mapToProduct(result.rows[0]);
      // Cache for 1 hour
      await this.cache.set(cacheKey, result.rows[0], 3600);
      return product;
    }

    return null;
  }
}
```

3. API Performance

Response Compression

```
import compression from 'compression';

app.use(compression({
  filter: (req, res) => {
    if (req.headers['x-no-compression']) {
      return false;
    }
    return compression.filter(req, res);
  },
  level: 6
}));
```

Rate Limiting

```
import rateLimit from 'express-rate-limit';

const apiLimiter = rateLimit({
  windowMs: 15 * 60 * 1000, // 15 minutes
  max: 100, // Limit each IP to 100 requests per windowMs
  message: {
    error: 'Too many requests from this IP, please try again later.'
  },
  standardHeaders: true,
  legacyHeaders: false,
```

```
});

app.use('/api/', apiLimiter);
```

🚀 Deployment Best Practices

1. Environment Configuration

Environment Variables

```
# Production environment
NODE_ENV=production
PORT=3000
DATABASE_URL=postgresql://user:password@host:5432/database
JWT_SECRET=your-super-secret-jwt-key
REDIS_URL=redis://host:6379
PAYMENT_API_KEY=your-payment-api-key
LOG_LEVEL=info

# Development environment
NODE_ENV=development
PORT=3000
DATABASE_URL=postgresql://localhost:5432/ecommerce_dev
JWT_SECRET=dev-secret-key
REDIS_URL=redis://localhost:6379
PAYMENT_API_KEY=test-key
LOG_LEVEL=debug
```

Configuration Management

```
class Config {
  static get databaseUrl(): string {
    return process.env.DATABASE_URL!;
  }

  static get jwtSecret(): string {
    return process.env.JWT_SECRET!;
  }

  static get isProduction(): boolean {
    return process.env.NODE_ENV === 'production';
  }

  static get logLevel(): string {
    return process.env.LOG_LEVEL || 'info';
  }
```

```
  static validate(): void {
    const required = [
      'DATABASE_URL',
      'JWT_SECRET',
      'PAYMENT_API_KEY'
    ];

    for (const env of required) {
      if (!process.env[env]) {
        throw new Error(`Missing required environment variable: ${env}`);
      }
    }
  }
}
```

2. Docker Configuration

Dockerfile

```
# Multi-stage build for production
FROM node:18-alpine AS builder

WORKDIR /app

# Copy package files
COPY package*.json ./
RUN npm ci --only=production

# Copy source code
COPY . .

# Build the application
RUN npm run build

# Production stage
FROM node:18-alpine AS production

WORKDIR /app

# Copy built application
COPY --from=builder /app/dist ./dist
COPY --from=builder /app/node_modules ./node_modules
COPY package*.json ./

# Create non-root user
RUN addgroup -g 1001 -S nodejs
RUN adduser -S nodejs -u 1001

# Change ownership
RUN chown -R nodejs:nodejs /app
USER nodejs
```

```
# Expose port
EXPOSE 3000

# Health check
HEALTHCHECK --interval=30s --timeout=3s --start-period=5s --retries=3 \
  CMD curl -f http://localhost:3000/health || exit 1

# Start application
CMD ["npm", "start"]
```

Docker Compose

```
version: '3.8'

services:
  app:
    build: .
    ports:
      - "3000:3000"
    environment:
      - NODE_ENV=production
      - DATABASE_URL=postgresql://postgres:password@db:5432/ecommerce
      - REDIS_URL=redis://redis:6379
    depends_on:
      - db
      - redis
    restart: unless-stopped

  db:
    image: postgres:14-alpine
    environment:
      - POSTGRES_DB=ecommerce
      - POSTGRES_USER=postgres
      - POSTGRES_PASSWORD=password
    volumes:
      - postgres_data:/var/lib/postgresql/data
    restart: unless-stopped

  redis:
    image: redis:7-alpine
    volumes:
      - redis_data:/data
    restart: unless-stopped

volumes:
  postgres_data:
  redis_data:
```

3. CI/CD Pipeline

GitHub Actions Workflow

```yaml
name: CI/CD Pipeline

on:
  push:
    branches: [ main, develop ]
  pull_request:
    branches: [ main ]

jobs:
  test:
    runs-on: ubuntu-latest

    services:
      postgres:
        image: postgres:14
        env:
          POSTGRES_PASSWORD: postgres
        options: >-
          --health-cmd pg_isready
          --health-interval 10s
          --health-timeout 5s
          --health-retries 5

    steps:
    - uses: actions/checkout@v3

    - name: Setup Node.js
      uses: actions/setup-node@v3
      with:
        node-version: '18'
        cache: 'npm'

    - name: Install dependencies
      run: npm ci

    - name: Run linting
      run: npm run lint

    - name: Run tests
      run: npm run test:coverage
      env:
        DATABASE_URL: postgresql://postgres:postgres@localhost:5432/test

    - name: Upload coverage
      uses: codecov/codecov-action@v3

  build:
    needs: test
    runs-on: ubuntu-latest
    if: github.ref == 'refs/heads/main'
```

```
    steps:
    - uses: actions/checkout@v3

    - name: Build Docker image
      run: docker build -t ecommerce-cart .

    - name: Push to registry
      run: |
        echo ${{ secrets.DOCKER_PASSWORD }} | docker login -u ${{
secrets.DOCKER_USERNAME }} --password-stdin
        docker tag ecommerce-cart ${{ secrets.DOCKER_USERNAME }}/ecommerce-
cart:latest
        docker push ${{ secrets.DOCKER_USERNAME }}/ecommerce-cart:latest

  deploy:
    needs: build
    runs-on: ubuntu-latest
    if: github.ref == 'refs/heads/main'

    steps:
    - name: Deploy to production
      run: |
        # Deployment script
        echo "Deploying to production..."
```

This best practices guide should be regularly updated and followed by all team members to ensure consistent, high-quality code and development practices.